

FutureKawa

Solution de suivi des stocks et des conditions de stockage (IoT)

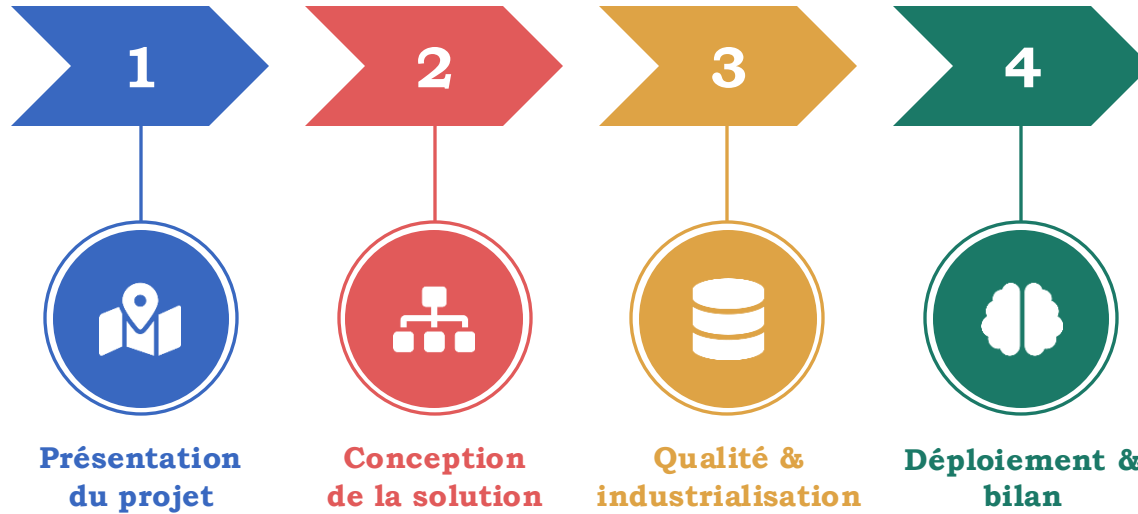
Dossier technique · Prototype avancé · Brésil · Équateur · Colombie

ÉLABORÉ PAR L'ÉQUIPE PROJET

Tarek Izerrouken · Sofiane Chelghoum · Yassine Moudjahid · Kelly Tsanga · Sara Haddad

Encadrante : Samah GHALLOUSSI

Plan



1



PARTIE I

Présentation du projet

Recueillir le besoin métier et cadrer le projet FutureKawa

FutureKawa : contexte & enjeux métier

FutureKawa est une entreprise internationale de caféiculture et de logistique de café vert, avec des plantations réparties au Brésil, en Équateur et en Colombie.

Le problème du stockage

- Conditions de stockage variables d'un entrepôt à l'autre, sans suivi fiable.
- Suivi encore semi-manuel : relevés papier, tableurs hétérogènes.
- Rotation FIFO peu fiable et preuves de conformité difficiles à produire.

CARTE D'IDENTITÉ

ACTIVITÉ

Caféiculture & logistique de café vert

IMPLANTATION

Brésil · Équateur · Colombie

CHAÎNE DE VALEUR

Production → lots → stockage → distribution B2B

COMMANDITAIRE

Directions des Opérations et de la Qualité

ENJEU

Qualité aromatique liée aux conditions de conservation

Problématique & objectifs du projet

Problématique Comment garantir et prouver, pour des clients exigeants, que chaque lot de café vert a été conservé dans de bonnes conditions, du stockage jusqu'à l'expédition ?



Centraliser le suivi

Suivi unifié des stocks par pays et par entrepôt.



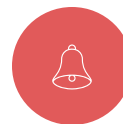
Garantir la traçabilité

Historique complet des lots depuis leur entrée en stockage.



Surveiller les conditions

Relevés IoT température / humidité en continu.



Détecter & signaler

Dérives de conditions et lots trop anciens.

Collecte des besoins métier

Entretiens semi-directifs avec les parties prenantes : exploitation, entrepôt, qualité, Supply Chain, DSI.



Centraliser le suivi

Interface web centralisée par pays et par entrepôt.



Automatiser les relevés

Capteurs IoT + broker MQTT, sans saisie manuelle.



Être alerté

Moteur d'alertes + e-mail : dérives et péremption.



Consulter l'historique

Historisation des mesures, courbes température/humidité.



Sécuriser l'accès

Authentification JWT + RBAC, isolation par pays.

Méthodologie de collecte des besoins

S

Préparation Questionnaire structuré en 4 catégories, avec les parties prenantes du projet

1

Contexte & problématiques Outils actuels (exploitation, entrepôt, qualité, siège) ; suivi manuel, réclamations, dérives non détectées

2

Besoins & contraintes Alertes automatiques, historique, vue multi-pays ; accès par rôle/pays, contraintes réseau terrain

3

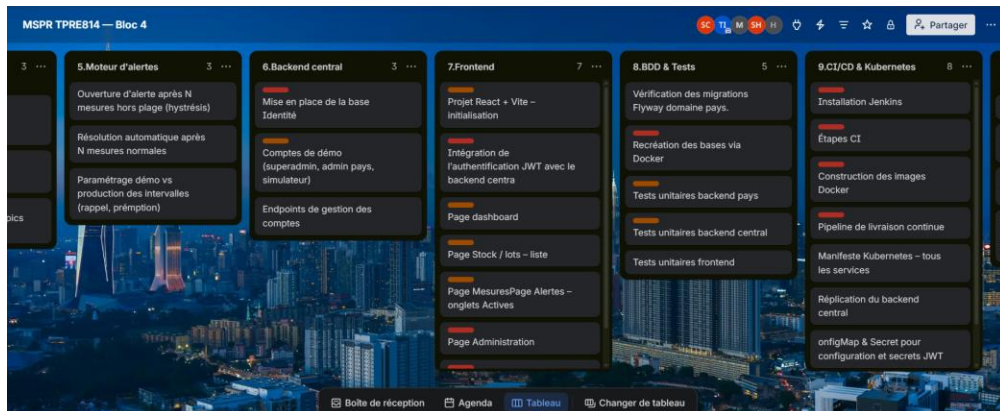
Retranscription & priorisation Tableau de synthèse besoin → fonctionnalité → priorité

R

Résultat Besoins fonctionnels prioritaires qui orientent l'architecture (partie II)

Organisation & gestion de projet

Un pilotage en méthode agile via un tableau Trello



Répartition des tâches

Colonnes Kanban : à faire, en cours, en revue, terminé

Priorisation

Backlog issu de la collecte des besoins (partie I)

Suivi & collaboration

Cartes assignées par membre ; équipe synchronisée en continu

Le tableau Trello matérialise l'organisation agile de l'équipe et la traçabilité de l'avancement du projet.

2

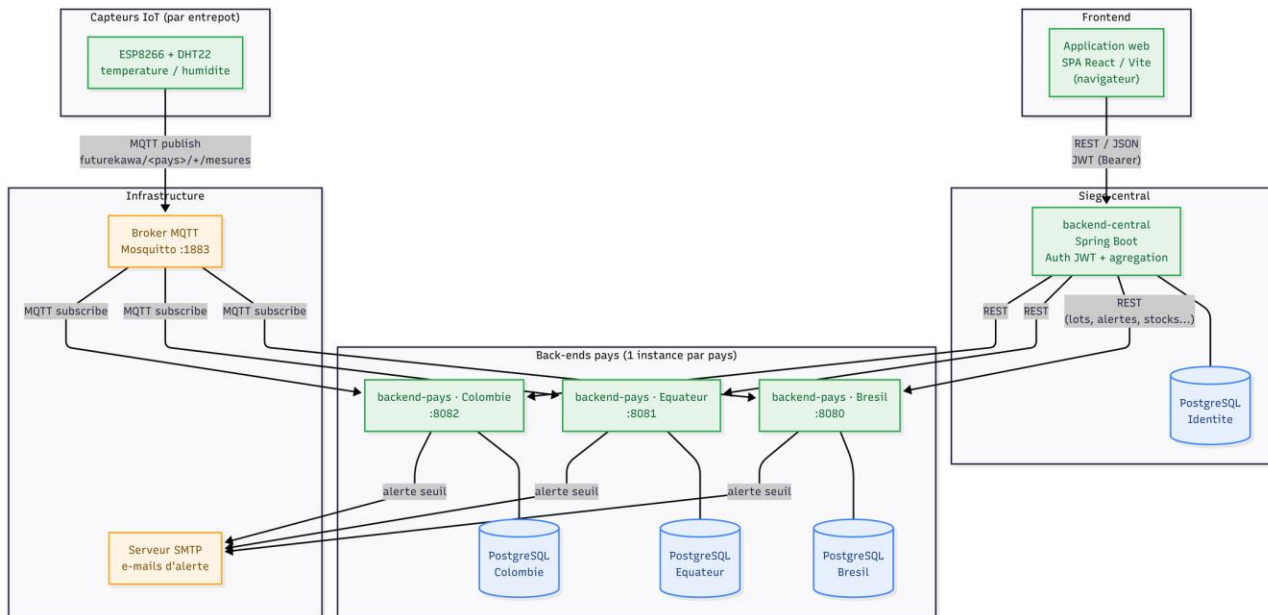


PARTIE II

Conception de la solution

Architecture globale distribuée et module IoT

Architecture globale distribuée



Composants et flux applicatifs

Architecture distribuée pays ↔ siège chaque pays dispose d'un back-end autonome, le siège consolide l'ensemble.



Back-end pays (×3)

Spring Boot / Java 21 —
ingestion IoT, API REST, moteur
d'alertes, notification e-mail.



Back-end central

Spring Boot — consolidation
multi-pays et authentification
JWT, sans donnée métier.



Front-end web

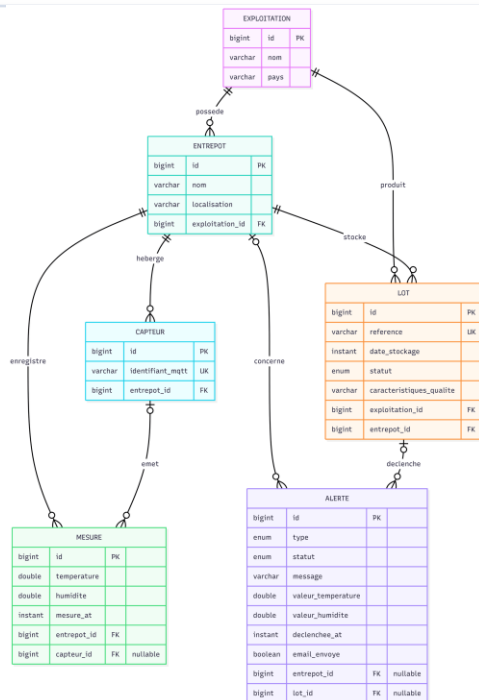
React / Vite (SPA) — stocks,
courbes
température/humidité, alertes.



Broker MQTT & SMTP

Eclipse Mosquitto (transport
IoT) + service d'envoi d'e-mails
d'alerte.

Deux flux principaux : montant (IoT → back-end pays) et consultation (front → central → pays), plus un flux d'alerte (back-end pays → SMTP).



Modèle de données du domaine Pays (IoT métier)

Comment le lire

Structure

Exploitation → Entrepot → Capteur : la hiérarchie physique du terrain.

Mesures

Le capteur émet des mesures température/humidité horodatées.

Lots

Un entrepôt stocke des lots ; chaque lot appartient à une exploitation.

Alertes

Une alerte concerne un entrepôt ou un lot précis (dérive, péremption).

Dispositif d'alertes — seuils par pays

Une alerte automatique est levée dans deux cas : conditions hors plage, ou péremption du lot.

Logique de déclenchement

- Conditions hors plage : température ou humidité hors bande acceptable.
- Péremption : lot stocké depuis plus de 365 jours.
- Tolérance : $\pm 3^{\circ}\text{C}$ en température, $\pm 2\%$ en humidité.
- Seuils paramétrés par profil pays.

29 °C

Brésil — idéal

31 °C

Équateur — idéal

26 °C

Colombie — idéal

365 j

seuil de péremption

Humidité idéale : Brésil 55 % · Équateur 60 % · Colombie 80 %

Logique de décision — anti-fausse-alerte

S

Mesure reçue Le back-end pays compare température et humidité aux seuils du pays

1

Anti-flapping & hystérésis Mesure isolée hors plage ignorée ; compteur d'anomalies incrémenté à chaque mesure consécutive

2

Ouverture 3 mesures hors plage consécutives → alerte ACTIVE + 1 e-mail

3

Rappel & acquittement Renotification toutes les 30 min si actif ; un opérateur acquitte → ACQUITTEE

R

Résolution 3 mesures normales consécutives → RESOLUE automatiquement

Sécurité, robustesse et tolérance aux pannes

Le back-end central centralise la sécurité ; plusieurs mécanismes assurent en parallèle la robustesse de la solution.

SÉCURITÉ

JWT · Authentification

Jeton d'accès 15 min +
refresh révocable

RBAC · Contrôle d'accès

Rôles porteurs de
permissions

Isolation par pays

Cloisonnement strict,
403 si refusé

Protection & audit

Mots de passe hashés,
actions consignées

ROBUSTESSE

Conteneurisation

Docker Compose,
orchestrable sur
Kubernetes

Auto-réparation

Sondes
liveness/readiness,
redémarrage
automatique

Reconnexion MQTT

Reconnexion
automatique après
coupure réseau

Anti-fausse-alerte

Hystérésis : pas
d'incident sur variation
passagère

MCU

NodeMCU v2 (ESP8266, ESP-12E)

Wi-Fi intégré, logique 3,3 V, faible coût, largement documenté.

CAPTEUR

DHT22

Température -40 à +80 °C, humidité 0-100 %, précision $\pm 0,5$ °C.

ÉCRAN

OLED SSD1306

0,96" 128×64 I2C — affichage local des mesures et de l'état.

Protocole MQTT — topics et pilotage à distance

S

Topic hiérarchique futurekawa/<pays>/+/mesures — le back-end pays s'abonne avec un joker

1

Payload & fréquence JSON {capteurId, température, humidité} ; fréquence paramétrable, plancher de 2 s

2

Pilotage à distance Configuration (topic, capteur, libellés) + commande start/stop

3

Découplage firmware Un même binaire sert n'importe quel entrepôt, la cible est injectée

R

Résultat déploiement simple et reproductible du module de terrain

Reconnexion et gestion des erreurs

La robustesse « terrain » du module IoT repose sur plusieurs garde-fous.



Reconnexion Wi-Fi

Relance de la connexion avec temporisation, sans blocage définitif.



Reconnexion MQTT

Connexion au broker vérifiée et rétablie à chaque cycle.



Lecture invalide

Valeur NaN du DHT22 ignorée, erreur affichée sur l'OLED.



Publication conditionnelle

Envoi uniquement si actif, configuré et topic renseigné.



Plancher de fréquence

Période minimale de 2 s pour protéger le broker.

Limites assumées dans ce prototype, et pistes identifiées pour la production.

Limites techniques

DHT22

Précision et cadence limitées (~0,5 Hz) : suffisant pour des dérives lentes.

Capteur unique

Point de défaillance unique par entrepôt, sans redondance.

Pas de tampon local Coupure réseau prolongée = mesures perdues, non mémorisées.

Risques & pistes

Sécurité MQTT

Broker en accès anonyme pour la démonstration

Dépendance Wi-Fi

Entrepôts mal couverts : prévoir un repli LoRa

Chiffrement

TLS à ajouter en production

Redondance

Second capteur par entrepôt à prévoir

Ces limites sont assumées : le livrable est un prototype avancé destiné à démontrer la faisabilité, pas une solution de production.

3



PARTIE III

Qualité et industrialisation

Plans de tests, intégration et déploiement continus

Stratégie de test

Tests unitaires automatisés ciblant la logique métier critique, isolés et rejouables.

Ce qui est ciblé

- Moteur d'alertes : seuils, hystérésis, péremption.
- Contrôle d'accès par pays.
- Consolidation multi-pays.
- Fonctions d'affichage du front-end.

16

tests back-end pays

6

tests back-end central

9

tests front-end

31

tests au total, 100 % verts

Dépendances externes simulées (Mockito) : ni base, ni broker MQTT, ni serveur démarré.

Cas de test — moteur d'alertes (back-end pays)

16 tests couvrent seuils, hystérésis et péremption.

Cas	Donnée d'entrée	Résultat attendu	Catégorie
Anti-flapping	1 mesure hors plage	aucune alerte levée	Hystérésis
Ouverture confirmée	3 mesures consécutives hors plage	1 alerte + 1 e-mail	Hystérésis
Péremption	lot stocké depuis > 365 jours	lot PERIME + alerte	Péremption
Anti-doublon péremption	alerte péremption déjà active	pas de seconde alerte	Péremption
Seuils — bornes incluses	valeur = borne	dans la bande ($\geq$ et $\leq$)	Seuils
Seuils — tout hors plage	température et humidité hors	hors bande	Seuils

Jeux de données construits dans les tests eux-mêmes : résultats prévisibles et vérifiables.

Cas de test — isolation et consolidation (back-end central)

6 tests couvrent l'isolation par pays et la consolidation multi-pays.

Cas	Donnée d'entrée	Résultat attendu	Portée
Super admin	rôle « tous pays » (*)	accès à n'importe quel pays	Accès
Admin de son pays	admin BR interrogeant BR	accès autorisé	Accès
Admin d'un autre pays	admin BR interrogeant EC	accès refusé (403)	Isolation
Fusion multi-pays	lots de deux pays	lots fusionnés, tagués pays	Consolidation
Tolérance aux pannes	un pays injoignable	pays ignoré, les autres renvoyés	Résilience
Pays inconnu	code pays invalide	erreur 404	Validation

Un administrateur rattaché à un pays ne peut jamais consulter les données d'un autre pays.

Gestion des anomalies — cycle qualité

S

Critères de réussite chaque cas valide ses assertions ; la suite complète (31/31) doit être verte

1

Constat & correction Test en échec ou dysfonctionnement signalé ; code corrigé, nouveau cas si besoin

2

Re-test & non-régression Suite rejouée localement, puis systématiquement en CI à chaque commit

3

Publication Résultats publiés au format JUnit, consultables dans Jenkins

R

Exemple le cas « anti-flapping » bloquerait toute régression future

Chaîne d'intégration continue (CI)

1

Checkout

Récupération du code depuis le dépôt Git.

2

Tests

31 tests automatisés, résultats publiés au format JUnit.

3

Packaging

Archives JAR des back-ends + build de production du front.

4

Images Docker

Construction des images conteneurisées des deux back-ends.

5

Archivage

Publication des artefacts téléchargeables depuis Jenkins.

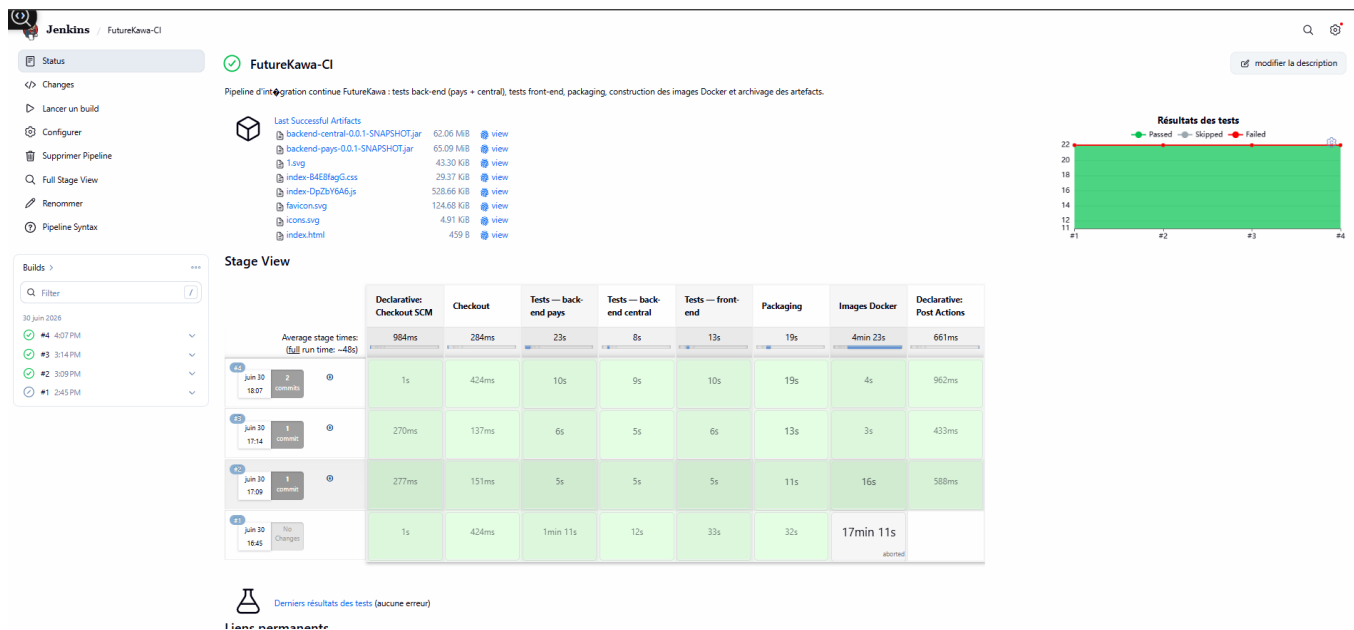
✓

Build réussi

En cas d'échec d'une étape, le build est en erreur et rien n'est publié.

Dockerfile dédié à la CI : réutilise le JAR déjà produit plutôt que de recompiler — construction ramenée de plusieurs minutes à quelques secondes.

Preuves d'exécution — intégration continue



Preuves d'exécution — déploiement Kubernetes

```
(base) PS C:\Users\ian_chel\Documents\GitHub\FutureKawa> kubectl get pods -n futurekawa-test
```

NAME	READY	STATUS	RESTARTS	AGE
fk-central-77fc4d895f-xz8r9	1/1	Running	4 (86m ago)	17h
fk-mosquitto-79c7b5fccd-djzmf	1/1	Running	5 (86m ago)	20h
fk-pays-br-766d646c4b-tz9wm	1/1	Running	3 (86m ago)	17h
fk-pays-co-7f6d48d77f-sfnht	1/1	Running	3 (86m ago)	17h
fk-pays-ec-6d6679f876-wkgct	1/1	Running	3 (86m ago)	17h
fk-postgres-57bdbcf94f-fvw26	1/1	Running	5 (86m ago)	20h

Tous les services Running ; redémarrages automatiques = auto-réparation.

Manifeste paramétrable décrivant l'ensemble du système, avec plusieurs mécanismes de résilience.

Mécanismes de résilience

Sondes liveness/readiness

Redémarrage automatique d'un conteneur défaillant.

Réplication

Le back-end central est répliqué (plusieurs instances).

ConfigMap / Secret

Configuration et secrets externalisés, séparés du code.

Limites & perspectives

Cluster local

Déploiement prouvé localement, pas encore en production

Cloud non provisionné

Ex. Azure AKS : prêt côté pipeline, non déployé

Déclenchement manuel

Webhook automatique sur push à ajouter

Sécurisation

Authentification et chiffrement du registre/broker à renforcer

Rolling updates sans interruption de service : les mises à jour progressives complètent le dispositif de résilience.

Démonstration technique — live

De la mesure capteur à l'alerte e-mail, en conditions réelles

1

Lancement de la stack docker compose up (3 pays + central + front + MQTT)

2

Connexion Login JWT sur la console web

3

Mesure IoT Le capteur ESP8266 publie température/humidité en MQTT

4

Alerte 3 mesures hors plage → alerte ACTIVE + statut du lot

5

Notification E-mail capturé dans Mailpit

Interprétation de code à l'appui : moteur d'hystérésis, filtre JWT, publish MQTT du firmware.



Merci de votre attention

FutureKawa · MSPR TPRE814 — Bloc 4 · EPSI · RNCP 35584